UNIVERSITY OF ELOUED | FACULTY OF EXACT SCIENCES

Department of Computer Science • Master 2 — AI & Data Science • 2026/2027

TP02 — How to Work with Big Data Files (+5GB)

Big Data Analytics | Experimental Laboratory Report

Dataset: Kitsune Network Attack Dataset — ARP_MitM_dataset.csv

Prepared by: Alya Toumi Group: 1 Date: 14 - 10 - 2026

This report records the actual experimental outputs supplied from the student's local Windows environment. Values have not been replaced by theoretical expectations.

1. Introduction and objectives

Loading multi-gigabyte CSV files can stress memory and increase processing time. This practical investigates conventional Pandas loading, streaming with Pandas chunks, and Dask's out-of-core task-based computation. The experiment records execution time and memory usage, then compares implementation behavior and results.

- Examine possible out-of-memory (OOM) limitations of standard pd.read_csv().
- Process a large CSV sequentially in chunks of 100,000 rows.
- Use Dask DataFrames, lazy evaluation, and compute-graph optimization.
- Compare runtime and observed memory footprint across methods.

2. Experimental environment and dataset

Item	Recorded configuration
Operating environment	Windows, Visual Studio Code, Python virtual environment (venv)
Installed libraries	pandas, dask[complete], psutil, matplotlib, pyarrow
Physical RAM	15.33 GiB usable/reported by psutil
Dataset	ARP_MitM_dataset.csv (Kitsune Network Attack Dataset, Kaggle)
CSV size	6.72 GiB (more than 5 GB)
Dataset dimensions	2,504,267 rows × 115 columns
Header / sample data	No header; first five rows inspected; columns inferred as float64 in sample
Numeric aggregation column	Column 0 (zero-based index)

The same source CSV was used throughout. Tasks 2 and 3 computed the global sum and mean of column 0. Data were read with header=None because the source has no header row.

3. Task 1 — The traditional failure (OOM challenge)

Method: The full CSV was loaded using pandas.read_csv(file_path, header=None). psutil measured memory within the data-loading process; an external Windows Job Object runner enforced a 3 GiB committed-memory limit on the process group and monitored its process tree.

Metric	Observed output
Status	Successful loading; no MemoryError occurred
Loaded rows / columns	2,504,267 / 115
RAM before loading	74.92 MB
RAM after loading	2,283.66 MB
CSV loading time	63.62 s
Total protected-run duration	75.04 s (includes a 10-second holding period)
Observed process-tree peak RSS	2,401.39 MB
Peak Windows Job committed memory	2,904.12 MB
Exit code	0

Interpretation: Although the CSV occupies 6.72 GiB on disk, the loaded numeric DataFrame required less memory than the serialized text. Reading succeeded even under the 3 GiB job committed-memory limit. Therefore, this run did not demonstrate a MemoryError. The course sheet asks to document such an exception; that outstanding demonstration is deferred pending consultation with the instructor. No failure should be claimed for the recorded run.

Peak RSS is the maximum sampled resident-set total across the observed process tree. Windows Job committed memory is a different metric and must not be treated as resident RAM.

4. Task 2 — Stream processing (Pandas Chunking)

Method: pandas.read_csv(..., header=None, chunksize=100000) iterated through the file. On each chunk, column 0 values were summed and counted; the global mean was computed from the accumulated sum and non-null count. The file was not retained as one DataFrame.

Metric	Observed output
Chunks processed	26 (25 × 100,000 rows plus 4,267 rows)
Total valid values	2,504,267
Global sum	791,809,853.843379
Global mean	316.184278
Execution time	64.01 s
RAM before execution	75.39 MB
Highest sampled process RSS	173.70 MB
Status	SUCCESS

RSS stayed near 166–167 MB during most full chunks, falling to 81.45 MB on the final smaller chunk. The result demonstrates approximately stable memory consumption during sequential stream processing. The 173.70 MB figure was sampled once per chunk and is not an exact continuous peak.

5. Task 3 — Dask: lazy and parallelized out-of-core compute

Method: dask.dataframe.read_csv read the same CSV lazily using blocksize="32MB" and header=None. The workflow selected column 0, built deferred sum and count expressions, optimized the calculation graph, then executed the two expressions with the threaded scheduler configured for two workers.

Metric	Observed output
Dask partitions	225
Workers / target block size	2 threaded workers / 32 MB
RAM before defining graph	104.84 MB
RAM after defining Dask DataFrame	107.62 MB
Original sum graph tasks	901
Optimized sum graph tasks	226
Total valid values	2,504,267
Global sum	791,809,853.843379
Global mean	316.184278
Compute time	97.25 s
Total execution time	97.44 s
Highest sampled process RSS	342.88 MB
RAM after execution	135.26 MB
Status	SUCCESS

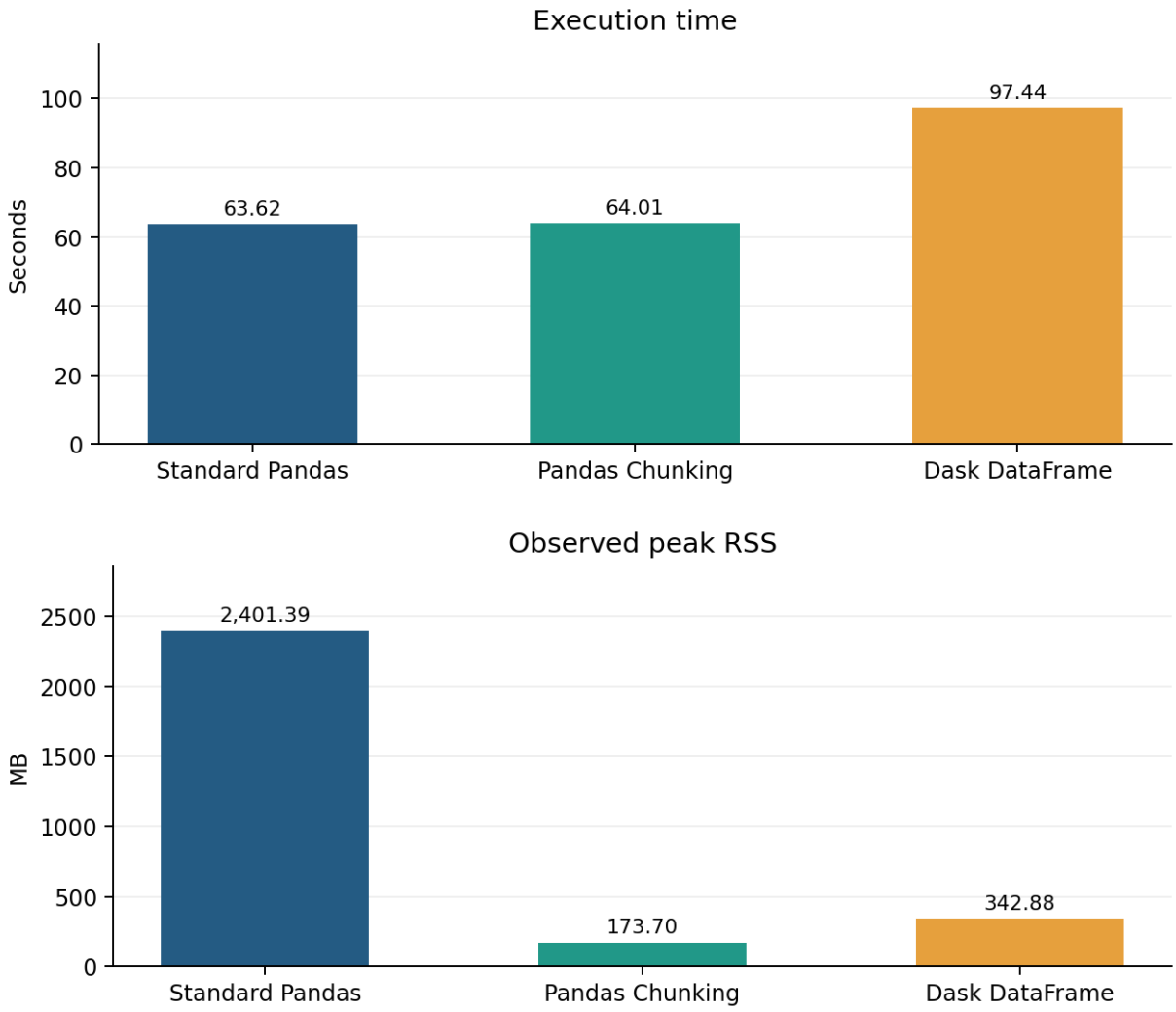
5.1 Lazy evaluation and compute-graph optimization

Lazy evaluation means that declaring a Dask DataFrame and composing sum/count expressions primarily constructs a graph of work; the full calculation is deferred until dask.compute() is called. RAM rose by only 2.78 MB between the pre-definition and post-definition measurements. This observation supports lazy loading but, by itself, does not establish that no metadata or sampling was read.

The sum graph decreased from 901 to 226 tasks after optimize(), a reduction of 675 tasks (approximately 74.9%). This demonstrates graph simplification, not necessarily faster execution: an unoptimized end-to-end runtime was not measured separately. Dask can schedule independent partition tasks concurrently, unlike the explicit sequential Pandas chunk loop.

6. Performance benchmark

Method	Time (s)	Observed RSS (MB)	Result
Standard Pandas	63.62	2,401.39	CSV loaded
Pandas Chunking	64.01	173.70	Sum + mean
Dask DataFrame	97.44	342.88	Sum + mean



6.1 Interpretation and limitations

The conventional Pandas test reported the shortest loading time, but its operation differed from the other tests: it loaded the CSV without aggregating column 0. Chunking had the lowest observed memory usage and nearly the same elapsed time. Dask returned the same numeric result as Chunking, but was slower and used more sampled RSS under this specific two-thread configuration. Coordination overhead, partition count, parsing cost, and workload characteristics can influence the result.

These are descriptive measurements rather than a fully controlled like-for-like benchmark. In particular, Task 1 RSS was measured across a process tree, whereas Tasks 2 and 3 sampled their main Python processes; RAM sampling cadence and computation workloads also differ. A strict comparison would standardize computations, repetitions, and process-tree peak monitoring.

7. Conclusions

All three tested methods successfully handled the selected CSV. Standard Pandas loaded the full file without OOM under the observed conditions. Pandas Chunking processed 2,504,267 values with low and approximately constant memory use, making it the most memory-efficient of the recorded executions. Dask demonstrated lazy operations, partitioned execution, and graph optimization, while

not outperforming Chunking on this workload. Choosing a tool therefore depends on RAM constraints, computation requirements, and measured—not assumed—performance.

8. Deliverables and outstanding work

- Python scripts: task1_pandas.py, safe_task1_runner.py, task2_chunking.py, task3_dask.py, benchmark.py.
- Results: task1_results.txt, task2_results.txt, task3_results.txt, benchmark_results.csv, execution_time.png, memory_usage.png.
- GitHub upload is required by the course handout; exclude the multi-gigabyte CSV itself and provide download/source instructions instead.
- Outstanding instructor question: whether a separate deliberately memory-limited demonstration of MemoryError is mandatory despite Task 1 successfully loading the full dataset.

Dataset source: https://www.kaggle.com/datasets/ymirsky/network-attack-dataset-kitsune

Course reference: TP02 — How to Work with Big Data Files (+5GB), University of Eloued, 2026/2027.